

Architecture

ENG1 Cohort 1 Team 3

Almira Demir
Mansi Deshkar
Laina Ha
James McDermott
Harry McLean
Vadim Mitko

University of York

Design Process

After creating the requirements, we had to make some decisions about the style and theme of our game before we could progress with creating an initial architecture, because the customer gave us a lot of freedom on those aspects of the product. The theme we decided on was retro-realism. We felt this would satisfy the non-functional requirement NFR_THEME because it is a popular style for video games of a similar nature that are played by similar demographics, such as older versions of The Sims. We also decided on a tile-based art style, because tiles integrate perfectly with the retro part of the theme, and they also simplify the implementation of the game, which will help to achieve the non-functional requirement NFR_PERFORMANCE.

We chose to use Responsibility Driven Design (RDD) to create the initial architecture for our game. Due to its focus on behaviours, we felt that it would give us freedom to abstract the low-level details for as long as possible while developing the design. This is because RDD brings the benefit of increased encapsulation by leaving the implementation of responsibilities to be private to each RDD object. It is also a well suited design technique for developing object-oriented programs (ideal as we are working with Java) as the technique makes good use of the inheritance hierarchy.

We began by creating a designer story based on the requirements that we had established and the style and themes we had decided upon. The goal here was to write a few sentences which encapsulated the requirements which would help us to all be on the same page with what we were creating, and to decrease friction during the design process. We settled on the following:

“We are developing a single-player 2D sandbox game where the user has 5 minutes to create a university campus by placing buildings on a predetermined map. The game is to be run as a desktop application running locally. The map will be tile-based and buildings can be placed on the tiles by the user. There will be a 5-minute timer that counts down. There will also be a counter for how many of each building category the user has placed.”

From here we began creating our first themes and candidate objects, and iterating on them, following the Responsibility Driven Design process using CRC cards (photos of these can be found on our [website](#)). During this stage, we were leaning towards a delegated control style, as it would allow us to keep our discussion at a higher level and we wanted to delegate work to objects that are more specialised to keep the design easy to understand. Once we were happy with our set of candidate objects, we began to map our candidates to classes and constructed our initial UML class diagram (Diagram 1), representing the overall structure of the project as we envisioned it at that point.

As shown in the diagram, we decided to use a closed, layered structure for our architecture, which takes the form of an inclusion tree in our class diagram. This gives one top-level object (Game) responsibility for everything in our application, and it handles this by dividing the work into smaller sections, and delegating them to smaller objects in the layer beneath, which it holds references to. The diagrams are provided with more details below (Diagram 1-2-3-4)

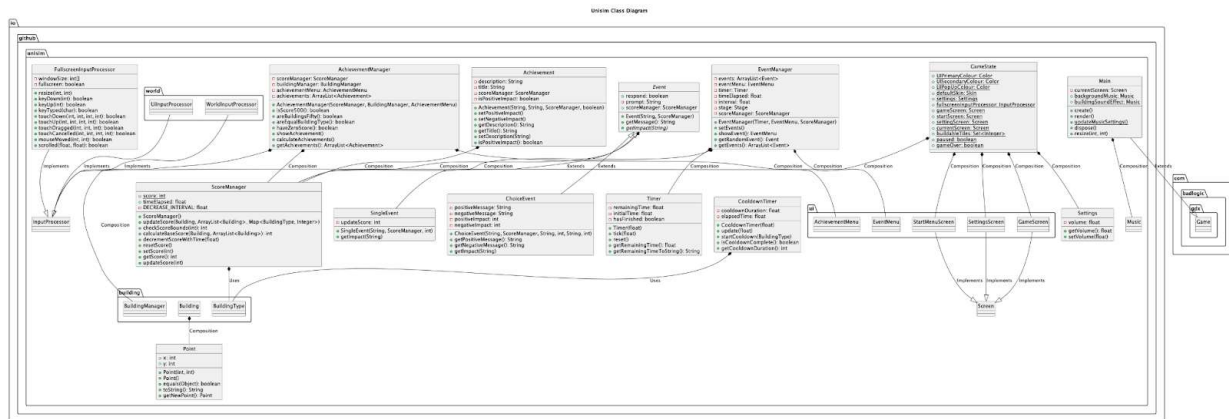


Diagram 1

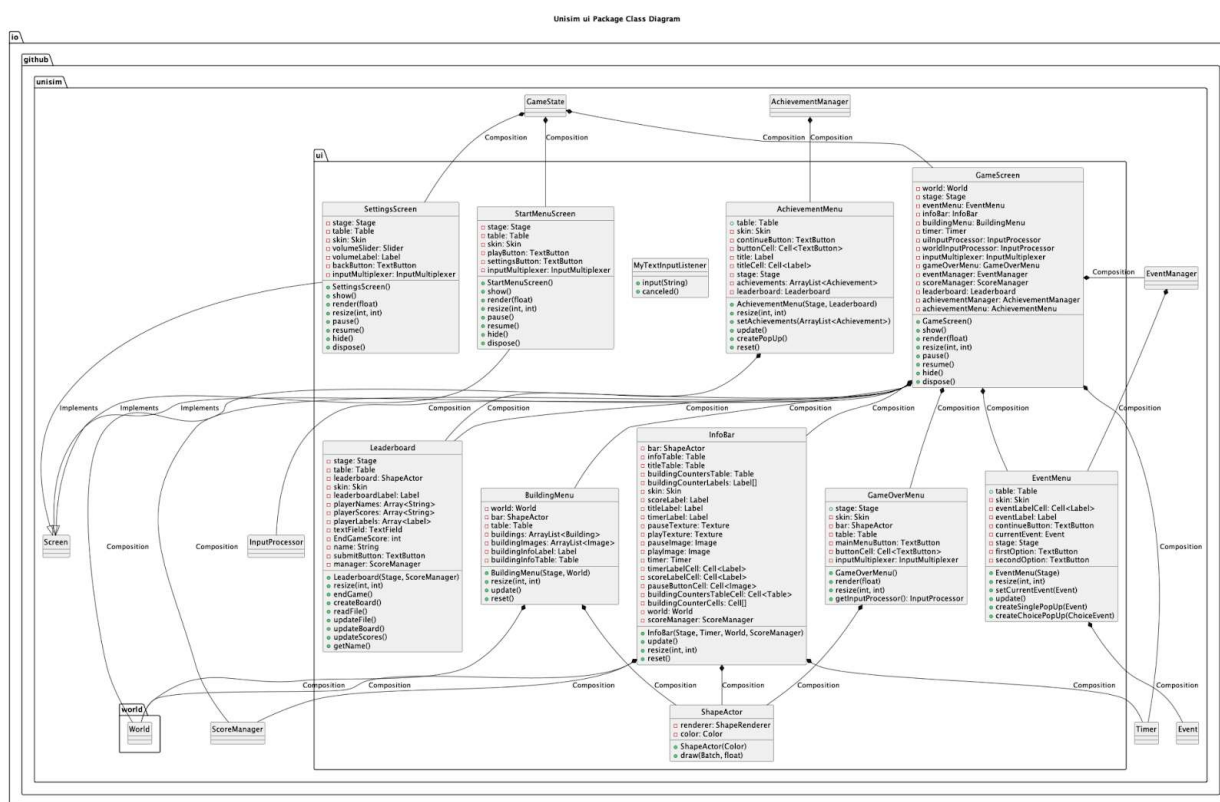


Diagram 2

Unisim building Package Class Diagram

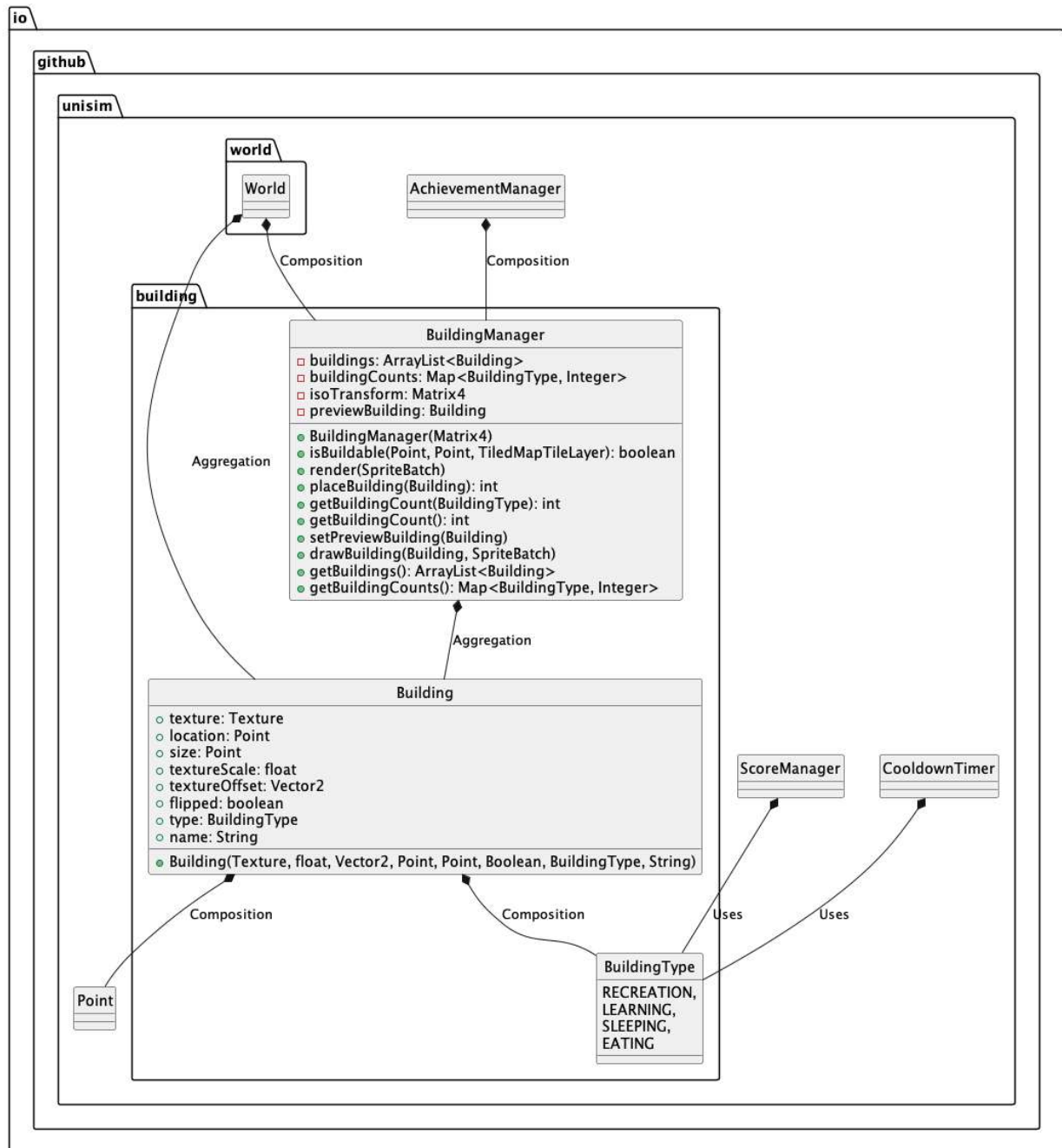


Diagram 3

Unisim world Package Class Diagram

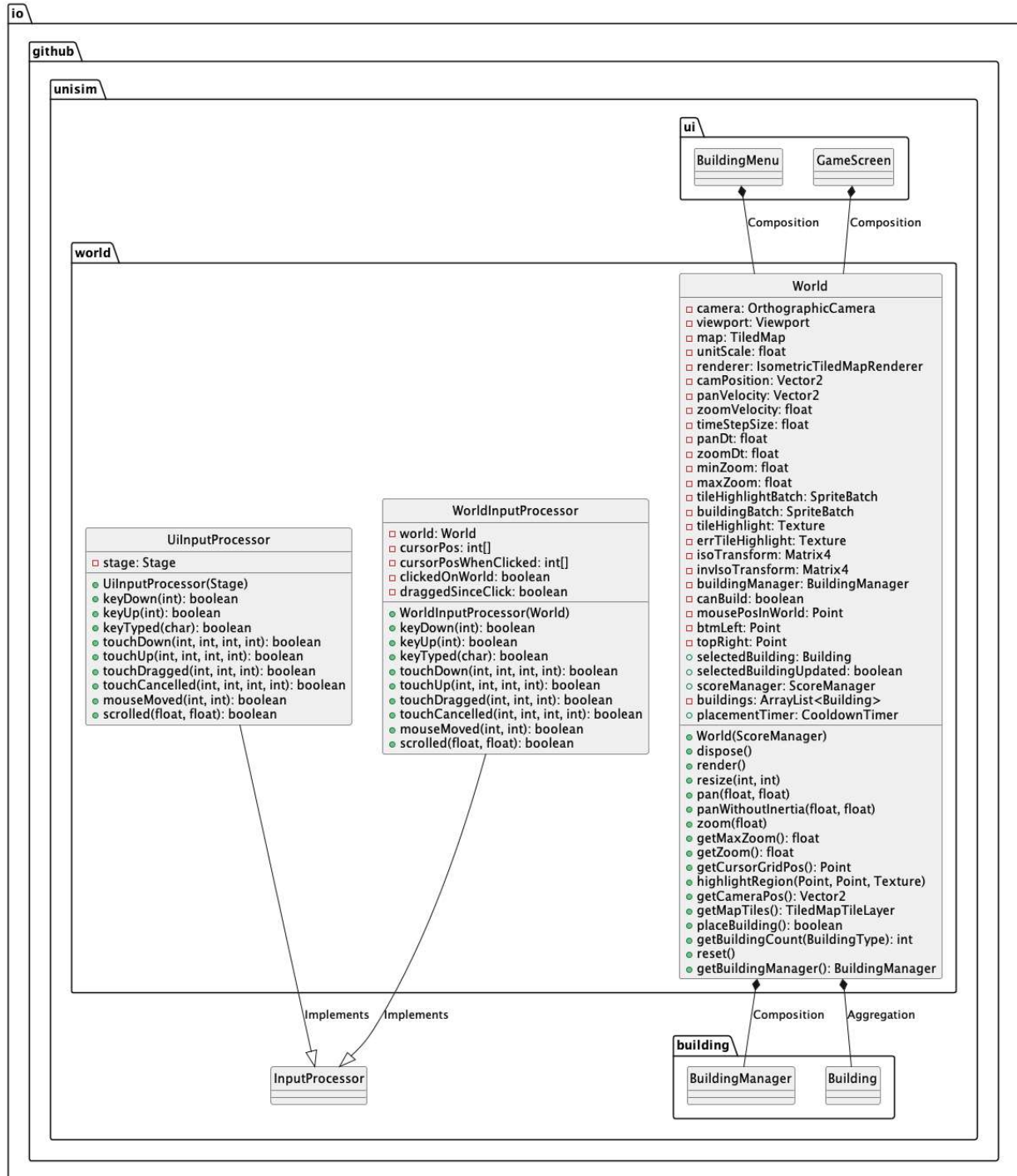


Diagram 4

We decided upon this structure because it encourages modular design, each object delegating its unrelated tasks to separate objects on a lower layer, and having no knowledge of other objects or systems not in the same subtree. By having objects not hold references to objects higher in the tree, we reduce bugs and keep the inclusions logical. Similar class structures are used in modern games and game engines, because the tree is easily scalable and it makes best use of the object-oriented paradigm.

Another advantage of this architecture is the ease of adding new features. Due to the nature of systems being separated into their own subtrees and not having access to the layers above, adding a new system can be done by creating the necessary classes, then inserting them into the right section of the tree. For example, if we were to add a player to the game,

we would create a class to manage the data and behaviour of the player, which we would then create a reference to from the world class (because the player lives within the world). We would then just modify the render method for the world and we would be done. The main advantage of this is that no other parts of the game would be affected, increasing code modularity and readability. This quality is needed to satisfy the non-functional requirement NFR_CODE_MODULARITY.

Before deciding on this structure, we rejected a lot of other concepts. One we considered heavily was the use of an entity component system. We recognised that the use of this type of system can be very advantageous in games, both in terms of speed, code structure and readability. The main benefit of this comes when there are a lot of different objects that reuse subsets of a set of components. We decided that in the case of our game, which has only a few types of objects (buildings, tiles and menus), using an entity component system would not be advantageous. Because there are so few objects, and in order for them to be implemented efficiently, their implementations will be quite dissimilar, the advantages of an entity component system would be very small, and not enough to outweigh the increased complexity introduced by such a complicated system in such a small game.

The above class diagram helps us to meet the following functional requirements:

FR_MAP - By the inclusion of the world class, which holds a collection of Tile objects, which make up the map.

FR_BUILDING - The building class will facilitate the placement of buildings.

FR_OBSTACLES - By the use of tiles, we can have certain tiles be non-buildable, which could then be used to make obstacles on the map.

FR_TIMER - The timer class will allow us to display a counting down timer to the user.

FR_BUILDING_COUNTER - Facilitated by the building counter class

Evolution of the Architecture

Implementation for the project lasted for about 3 weeks. For the first week or so, we followed the initial architecture class diagram quite closely. As we were working with the libGDX game engine, there were a few things that we changed, such as making use of the libGDX classes TiledMap and IsometricTiledMapRenderer. This eliminated the need for our own Tile class, and reduced the complexity of the world class, which improved the conciseness of the code, and harnessed the well-optimised code provided by libGDX. This helps us work towards requirements NFR_CODE_MODULARITY and NFR_PERFORMANCE respectively. To reduce confusion, we also renamed the top level Game class to Main, as the top level libGDX class that Main/Game needs to extend is already called Game.

When we came to implementing the menu interfaces for the game, it became clear that, in order to make optimal use of libGDX's menu framework (scene2d-ui), we would need to deviate slightly from our original application structure. This is because of the way scene2d provides multiple menus - the Screen interface. This would allow us to easily switch between different menus and the game screen without having to reload each menu before every switch. In order to take full advantage of this, it made sense for the World object to be held by a Screen Class, rather than the game class, and then make the Main class hold a reference to all the different screens.

Quite soon after we began to implement the idea above, we encountered a problem with this change to our architecture: it became necessary for the individual implementations of the Screen interface to be able to modify the current screen that is being displayed to the user. For example, if you are currently being shown the startup screen of the game, and you press

play, the screen needs to detect that click, then somehow notify the Main class that it needs to switch the current screen to the game screen. This conflicts with our ideal tree structure in which each object has no knowledge of the objects that hold references to it.

At this point, it became necessary to research how more established libGDX games accomplish this. Taking inspiration from the game Mindustry, which has a class called Vars¹ that holds all the information about the current state of the game as static members, we created a class called GameState which holds information such as the current screen and whether or not the game is paused as static member variables. This allows those fields to be accessed similarly to global variables by any code that needs to. It is important that we make only minimal use of this class, as overdependence on global variables can lead to code that is very difficult to trace.

With these new changes, we were able to continue smoothly to the end of the current development stage, implementing all the features required for this assessment. The tree structure is now split into individual screen classes and the state of the game is stored as static members of a class. This was necessary because we are working with the event-based libGDX in combination with our own layered architecture. This highlights how it is not necessary to conform perfectly to any specific architectural concept, but to take the best bits of all of them to suit our needs, while still leaning more towards one (layered). The final structure of the architecture is presented below (Diagram 2) and it can be found [here](#) with high resolution.

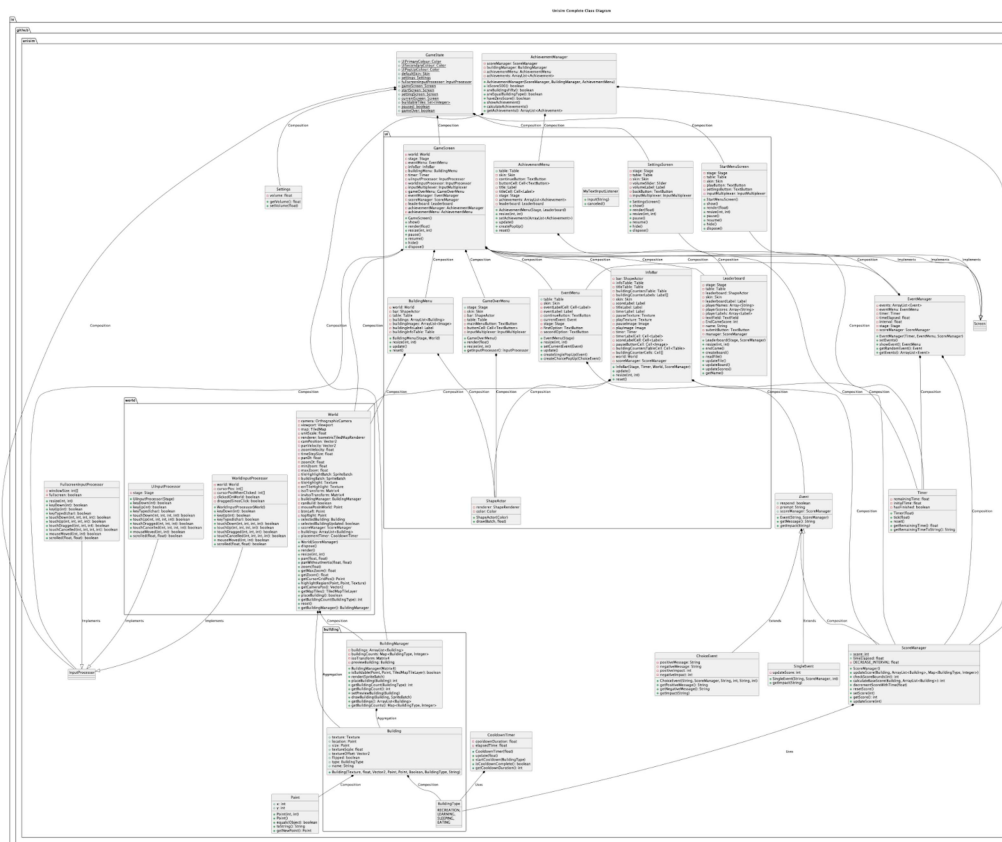


Diagram 5

¹ "Anuken/Mindustry," *GitHub*, 2017.
<https://github.com/Anuken/Mindustry/blob/master/core/src/industry/Vars.java> (accessed Nov. 07, 2024).

Behavioural Diagrams

We created a state diagram to show the different game states that the program can be in throughout the gameplay. The idea of this diagram is to show all the different menu screens at a high level. (Diagram 6)

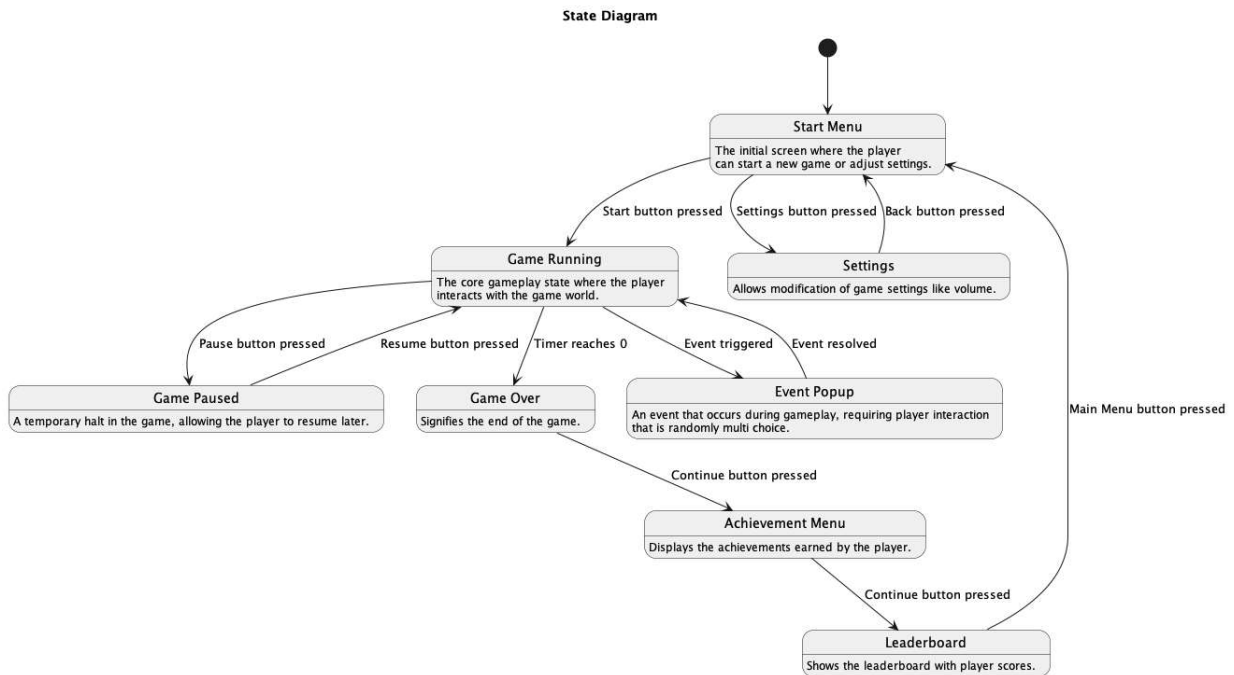


Diagram 6

We also created sequence diagrams to show the main actions that can happen during gameplay and relate these to our architecture. The most important (complex) behaviours that can be invoked by the payer are placing a building and traversing the map. The UML sequence diagrams for each can be seen in Diagrams 7 and 8 respectively.

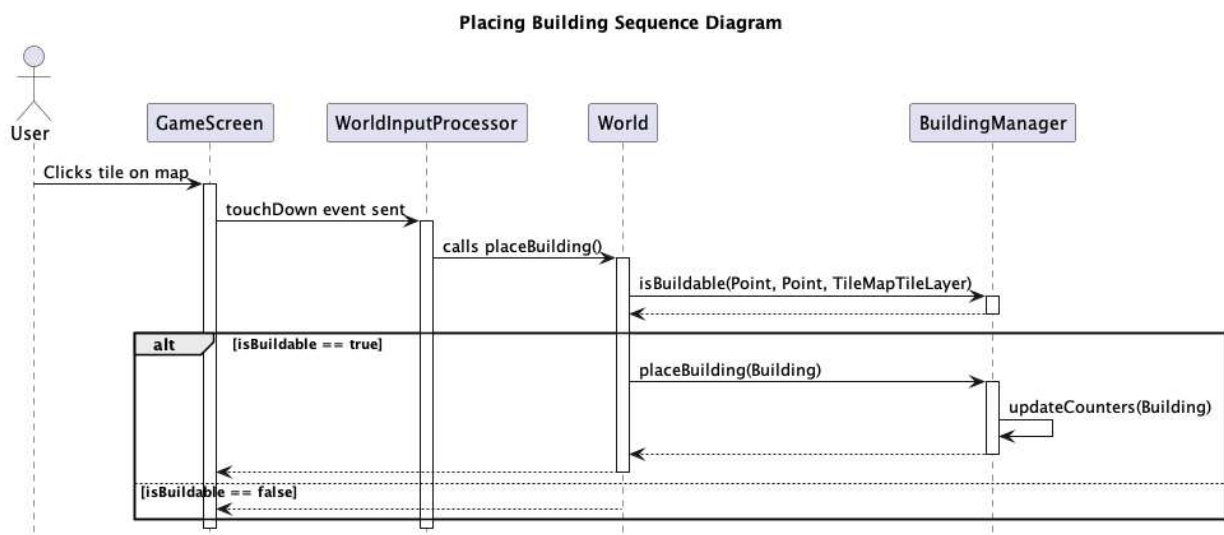


Diagram 7

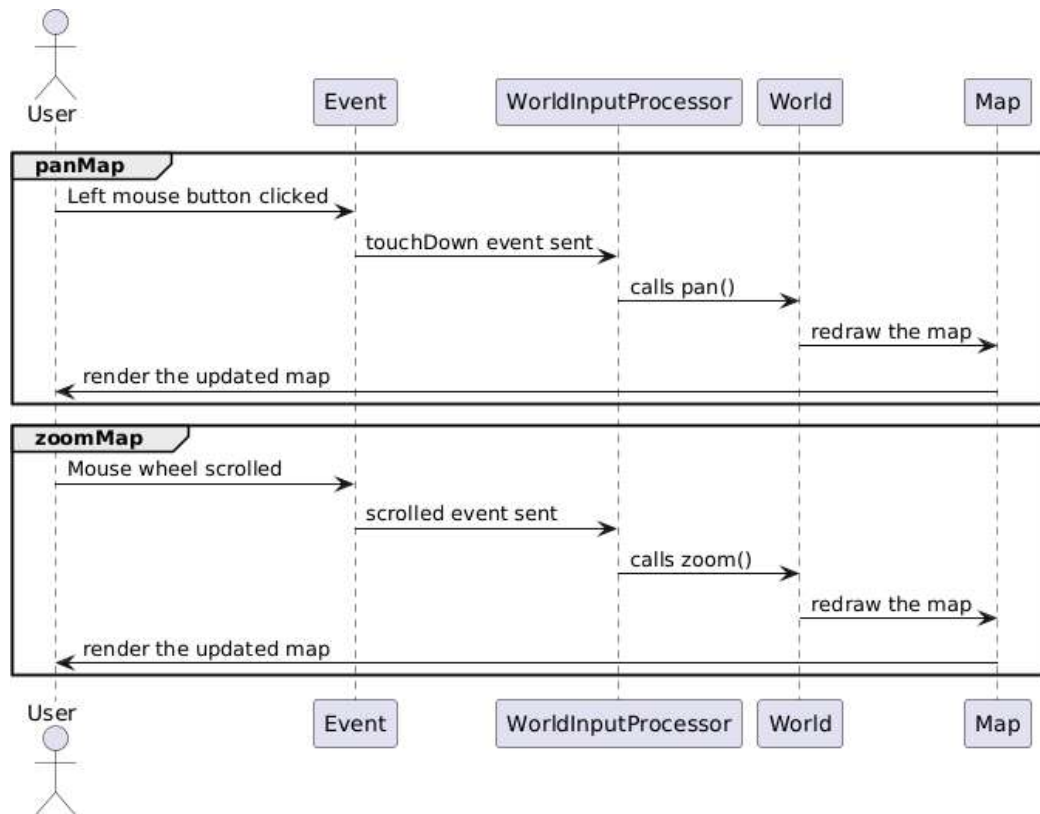


Diagram 8

We created a sequence diagram for events during the game. While the timer is running a random Event, either a SingleEvent or a ChoiceEvent pops up and it may have positive or negative impact on the satisfaction score. The effect of SingleEvent is predetermined, while that of ChoiceEvent is dependent on the user's choice. Here is the sequence diagram for events (Diagram 9).

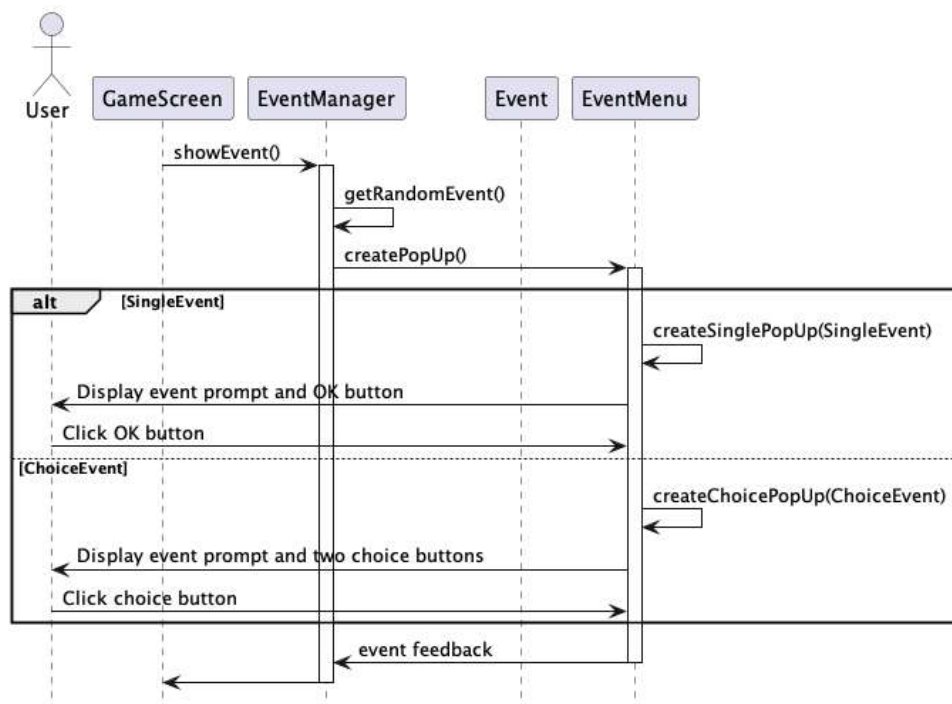


Diagram 9

Lastly we created an achievement displayer and a leaderboard which can be seen after the game is ended. Also we made some enhancements for building placement and map traversal.

Diagram Creation

We chose to create our architectural diagrams using the class diagram, sequence diagram and state diagram sections of PlantUML². This is beneficial because the diagrams can be specified using text, then rendered dynamically using an interpreter. This suited our needs very well as the text-based sources for our diagrams integrated very well with Git, which was our chosen version control solution. This was important due to our use of an agile software development methodology, which necessitated evolving the structure of the project, and therefore the diagrams over time as we moved through development. The diagrams were rendered for this document using the online server on PlantUML.

² "PlantUML" *PlantUML.com*. <https://www.plantuml.com/plantuml/uml/>